Project Presentation of IOHE

On

Real-Time Collaborative Development Environment

Durgesh Kumar 2210991551

Supervised By: Dr. Shikha

Department of Computer Science and Engineering,

Chitkara University, Punjab

Introduction

2.1 Background

- **Lack of Instant Feedback**: Remote collaboration often lacks the immediate feedback available in physical spaces, leading to team disconnection.
- **Over-dependence on AI**: Current editors either offer no help or too much AI assistance, which hinders students' logical skills and "hand practice".No transparent bidding mechanism
- **Platform Fragmentation**: The absence of integrated messaging forces users to switch between multiple platforms for coding and communication.
- **Inefficient Troubleshooting:** There is a lack of unified environments where real-time execution and team communication happen simultaneously

2.2 Problem Statement

- **Develop a Unified Collaborative IDE**: Create a MERN-stack platform that enables real-time code synchronization with built-in Chat and Voice tools.
- **Implement AI Activation Logic**: Design a system where AI assistance is locked and only triggers upon compilation failure to ensure independent learning.Provide automated permit notifications
- **Secure Multi-User Environment**: Utilize UUID-based rooms to facilitate secure, synchronized coding sessions and real-time audio interaction.
- **Foster Technical Discipline**: Limit AI usage to troubleshooting to ensure users focus on developing logical reasoning and independent problem-solving skills.

Software and Hardware Requirement Specification

CHITKARA UNIVERSITY

- **3.1 Methods**
- Real-Time Collaborative Development Environment is developed using Agile methodology , focusing on iterative development and constant testing of core modules. The project is structured into multiple sprints to implement real-time synchronization, integrated voice chat, and the conditional AI debugging engine. This continuous integration approach ensures the platform's reliability, security, and high-performance scalability. By maintaining this modular focus, the system effectively bridges the gap between independent learning and seamless remote collaboration.
- **3.2 Programming/Working Environment**
- **Programming Language**: React.js (Vite) with Monaco Editor for high-performance code editing.
- **Backend & Email Service** : Node.js and Express.js for server-side logic and safe code execution.SendGrid API for secure and automated workspace invitation system.
- **Real-Time & Voice**: Socket.io for bi-directional syncing and Agora RTC for audio communication.
- **AI Integration**: Grok AI for conditional error troubleshooting.
- **Architecture:** Stateless Real-Time Client-Server Model.
- **3.3 Requirements to run the application**
- **Operating System:** Windows 10 / Linux / MacOS
  RAM: Minimum 8GB
  Processor: Intel i5 or equivalent
  Software Dependencies: Node.js, npm, Supabase configuration, Clerk setup
  Browser: Latest versions of Chrome, Firefox, or Safari

Keywords

- Collaborative IDE, Real-time Synchronization, MERN Stack, Socket.io, Agora RTC, Voice Chat Integration, Conditional AI Debugging, Grok AI, Monaco Editor, Remote Pair Programming, Software Development Environment, Node.js Child Process, UUID-based Room Management, SendGrid Email Invitations, Technical Discipline, Independent Learning.

TECHNOLOGIES AND TOOLS

Real-Time Collaborative Development Environment utilizes a cutting-edge tech stack to ensure high performance, security, and real-time collaboration.

React.js (Vite): Powering a responsive frontend and synchronized code editing via the Monaco Editor.

Socket.io: Facilitates bi-directional, low-latency communication for real-time code and chat syncing.

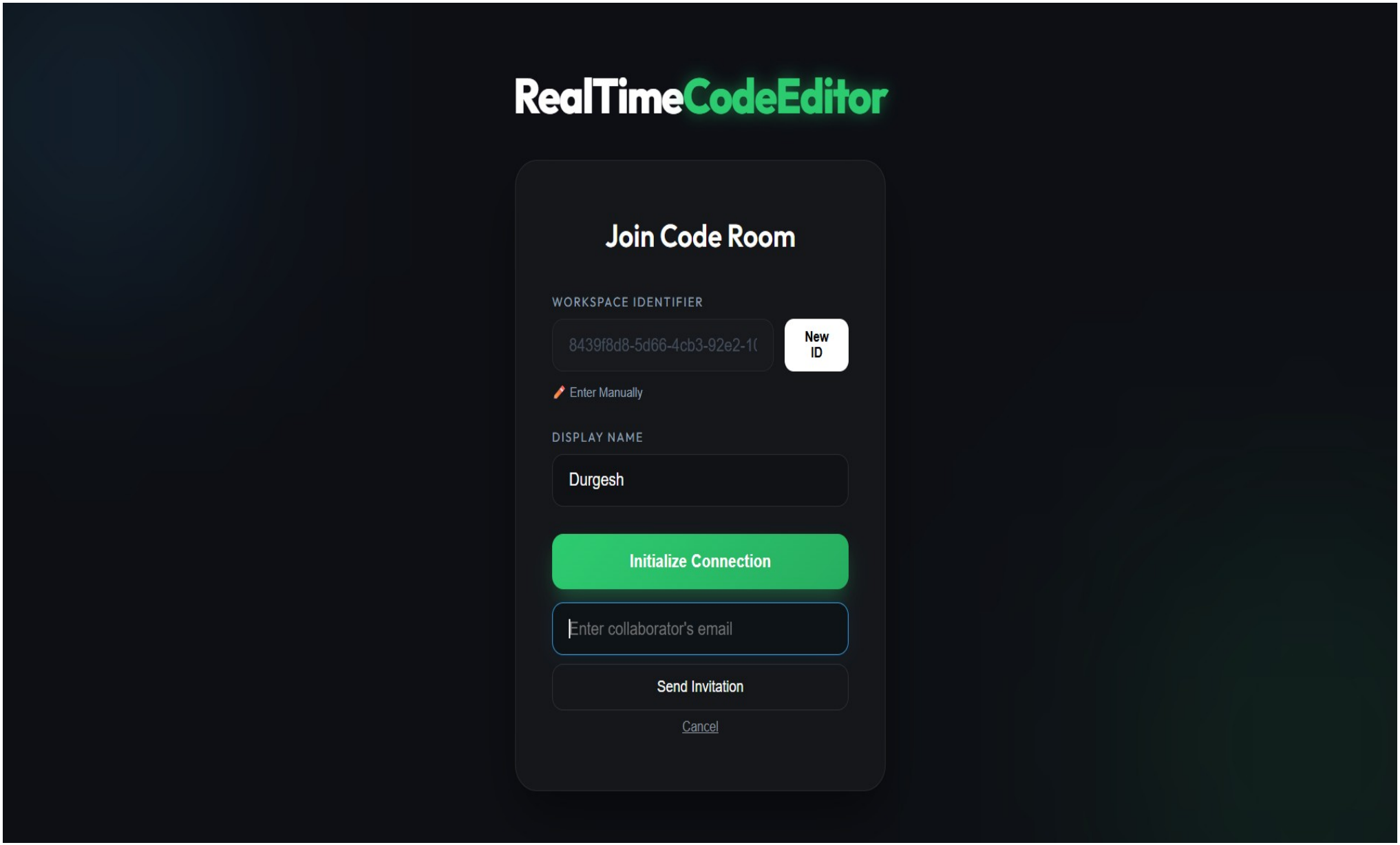
Node.js & Express: Handles backend logic and secure multi-language code execution via child processes.

Agora RTC: Provides high-quality, real-time voice chat within collaborative rooms.

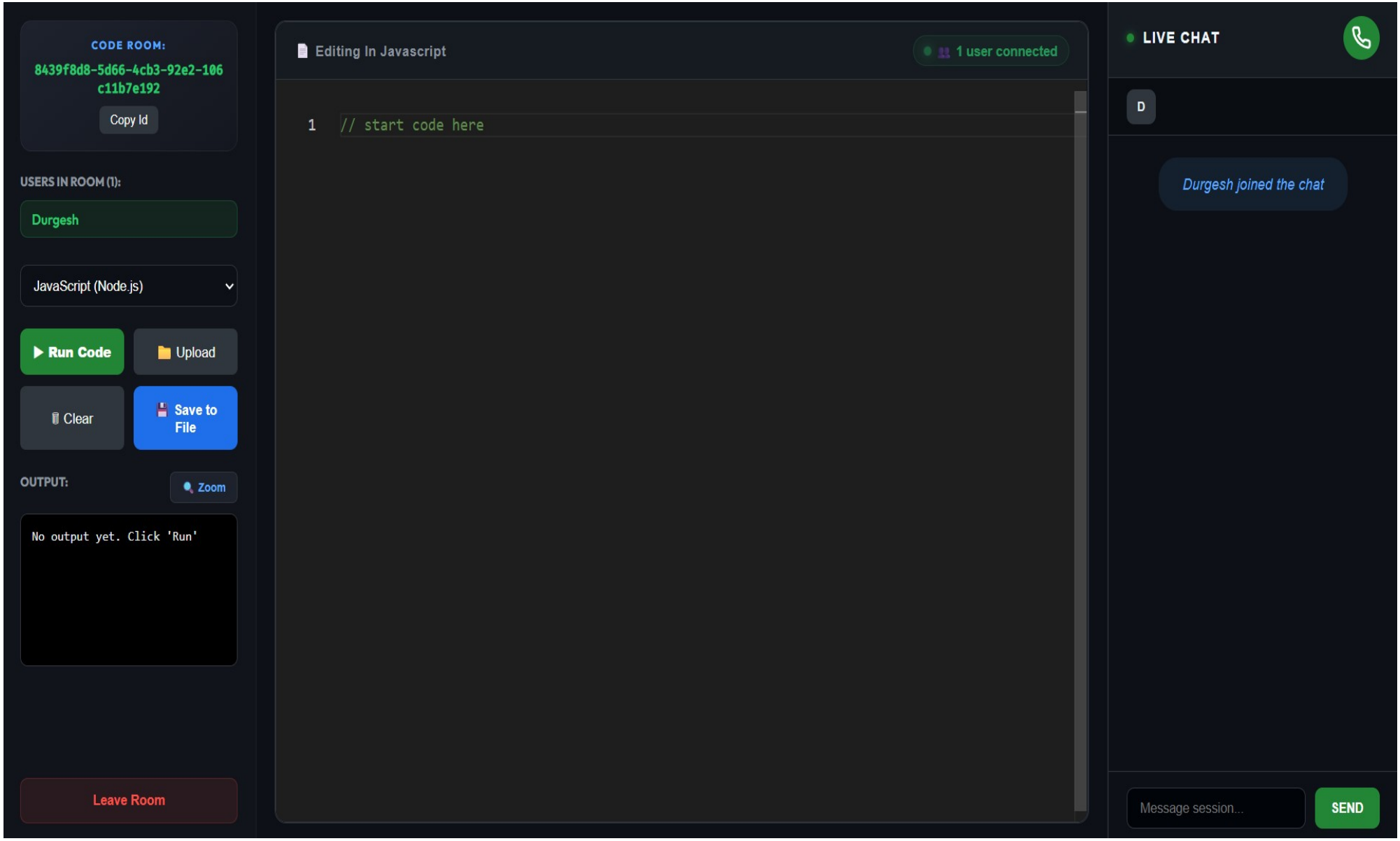
Grok AI: Powers the Conditional AI Debugger, which unlocks mentorship only during compilation failures.

SendGrid API: Manages the automated workspace invitation system via secure email delivery.

VISUAL REPRESENTATION



VISUAL REPRESENTATION



VISUAL REPRESENTATION

CHITKARA UNIVERSITY

CODE ROOM:
8439f8d8-5d66-4cb3-92e2-106c11b7e192
Copy Id

USERS IN ROOM (1):
Durgesh

JavaScript (Node.js)

Run Code | Upload | Clear | Save to File

OUTPUT: Zoom

```
[7:07:08 AM] - Run by: Durgesh
❌ ERROR:
file:///opt/render/project/src
console.log(5+"")}
                ^

SyntaxError: Unexpected token
```

Fix with AI

Leave Room

Editing In Javascript

1 user

```
// start code here
console.log(5+"")}
```

$_ Console Output

```
[7:07:08 AM] - Run by: Durgesh
❌ ERROR:
file:///opt/render/project/src/Backend/temp/a4e9a5bd-1679-4d98-9df4-455dd3b437a5.js:2
console.log(5+"")}
                ^

SyntaxError: Unexpected token '}'
    at compileSourceTextModule (node:internal/modules/esm/utils:346:16)
    at ModuleLoader.moduleStrategy (node:internal/modules/esm/translators:107:18)
    at #translate (node:internal/modules/esm/loader:546:20)
    at afterLoad (node:internal/modules/esm/loader:596:29)
    at ModuleLoader.loadAndTranslate (node:internal/modules/esm/loader:601:12)
    at #createModuleJob (node:internal/modules/esm/loader:624:36)
    at #getJobFromResolveResult (node:internal/modules/esm/loader:343:34)
    at ModuleLoader.getModuleJobForImport (node:internal/modules/esm/loader:311:41)
```

AI Assistant

Close

EXPLANATION:

The error occurs due to a syntax error caused by an unexpected closing bracket '}' at the end of the console.log statement. The correct syntax for the console.log statement should not include the closing bracket at the end.

FIXED CODE:

```
console.log(5+""
)
```

✨ Replace My Code

Future Scope

Real-Time Collaborative Development Environment aims to evolve from a collaborative editor into a comprehensive AI-driven development ecosystem with the following planned enhancements:

AI-Powered Code Optimization: Beyond debugging, the AI will provide logic optimization suggestions to improve code efficiency and performance.

Automated Logic Flowcharts: Integration of a module that automatically generates visual flowcharts from user-written code to help in better architectural understanding.

Integrated Video Conferencing: Expanding from Agora-based voice chat to full high-definition video calls and screen sharing within the workspace.

Advanced Version Control: Implementing a built-in "Git-like" history system allowing users to track changes and revert to previous code states seamlessly

.

Multi-user Pair-Programming Analytics: An AI dashboard to analyze team collaboration patterns and provide insights into coding productivity.

Conclusion

The evolution of the **Real-Time Collaborative Development Environment** represents a significant advancement in collaborative platforms. By bridging the gap between traditional editors and modern remote pair-programming needs, the system integrates real-time code synchronization, integrated voice communication, and an intelligent AI activation logic into a single web-based platform. It enables developers to collaborate with transparency, foster teamwork, and execute code with high precision.

Built using the **MERN stack, Socket.io, and Agora RTC**, this environment provides a seamless and scalable experience with optimal performance and security. The platform not only simplifies the debugging process through its unique **Conditional AI Debugger** but also ensures that users retain their logical reasoning and "hand practice" by limiting AI assistance specifically to troubleshooting phases.

In summary, the **Real-Time Collaborative Development Environment** is more than just a code editor; it is an integrated educational and collaborative ecosystem that promotes technical discipline and trust within development teams. With ongoing improvements like AI-driven code optimization and visual logic flowcharts, the platform is poised to become a true one-stop solution for intelligent and ubiquitous digital software management.

Bibliography/References

React (Official Documentation)
Meta. *React – A JavaScript Library for Building User Interfaces.*
https://react.dev
Vite (Official Documentation)
Evan You. *Vite – Next Generation Frontend Tooling.*
https://vitejs.dev
Node.js (Official Documentation)
OpenJS Foundation. Node.js – JavaScript runtime built on Chrome's V8 engine.
https://nodejs.org
Express.js (Official Documentation)
OpenJS Foundation. Express – Fast, unopinionated, minimalist web framework for Node.js.
https://expressjs.com
Socket.io (Documentation):
Socket.io – Bidirectional and low-latency communication for real-time applications.
https://socket.io/docs
Agora RTC SDK:
Agora.io. Real-Time Engagement Platform for Voice and Video.
https://docs.agora.io
Monaco Editor:
Microsoft. The code editor that powers VS Code.
https://expressjs.com
SendGrid API:
Twilio SendGrid. Email Delivery Service for automated invitations.
https://docs.sendgrid.com

Thank You